MATH70120 Coursework

Anandakrishnan Suresh[*], Eugene Lee[†], Rishi Massey[‡], Zareen Hyatt[§]

February 2026

Abstract

We tackle the problem of optimal execution in finance by extending the work of Hendricks and Wilcox [1], which formulates execution as a finite-horizon Markov Decision Process anchored to the Almgren-Chriss (AC) framework. The original study applies tabular Q-learning to L2 order book data. We clarify the modelling choices and place them in context of standard execution literature. We also contrast them with recent deep-RL execution work, where differences in simulators and impact assumptions often prevent meaningful cross-paper comparisons [2, 3, 4]. We then re-implement the Hendricks-Wilcox environment on AAPL L2 data and run controlled experiments. We fix the execution simulator, reward, feature discretisation, and an AC-multiplier action space. We compare the original paper implementation of tabular Q-learning against extended implementations of SARSA, Double DQN, and PPO under two regimes: *by-hour* (tabular: one policy per start time H; deep RL: one policy trained on the union of by-hour starts) and *no-hour* (single pooled policy; uniform episode starts). Hour-conditioned tabular methods often improve over tuned AC on a per-hour median basis, but the gains are not uniform across hours or horizons. Deep RL often converges to near-deterministic AC-like policies, including exact collapse to AC in some settings. Its best pooled gains occur at longer horizons, and are better explained by variance and tail-risk reduction than by learning a rich state-dependent execution rule. Our contributions are two-fold. First, we provide an apples-to-apples comparison of RL-driven optimal execution of tabular, deep value-based, and actor-critic methods under one L2-driven execution simulator. Second, we identify design choices that drive the results in practice: time conditioning, terminal completion, and the choice of aggregation.

1 Introduction

In this section, we introduce optimal execution as a Markov Decision Process, then proceed to bridge it to the Almgren-Chriss (AC) algorithm to ground [1] as our main framework. We clarify and compare the price process and execution mechanics assumptions of the main work and related work. Optimal execution (liquidation or acquisition) is a day-to-day problem for traders who execute on behalf of their clients. If a client asked one to sell or buy 10,000 TSLA stocks, the trader would have to execute it within trading hours.

[*]CID 06042471
[†]CID 06049056
[‡]CID 02218595
[§]CID 06055591

Optimal execution as a finite-horizon MDP We model intraday optimal execution of a parent order of size V over a finite horizon $t = 1, \ldots, T$ as an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, T)$ with $\gamma = 1$. Note that discount is set to 1 to simplify the finite time horizon.

State At time t, the agent observes

$$s_t = (t,\ q_t,\ x_t) \in \mathcal{S}, \tag{1}$$

where q_t is remaining inventory and x_t denotes observable market features (e.g., spread, depth/volume, imbalance).

Action The action $a_t \in \mathcal{A}(s_t)$ specifies the execution decision over $[t, t+1]$, commonly as shares to trade, or an equivalent trading rate, subject to $0 \le a_t \le q_t$.

Dynamics Inventory evolves as $q_{t+1} = q_t - \tilde{a}_t$, where $\tilde{a}_t$ is the executed quantity resulting from the submitted action a_t. In our simulator $\tilde{a}_t$ can be smaller than a_t due to a participation cap, while market features evolve stochastically:

$$x_{t+1} \sim P(\cdot \mid x_t, a_t), \tag{2}$$

with P either exogenous (no market-impact feedback) or endogenous (including permanent impact).

Reward and objective. We follow the sign convention of [1] and use (signed) implementation shortfall (IS) versus the arrival price as the episode return, so our objective is to maximise expected cumulative reward over the horizon. For both BUY and SELL orders, *worse-than-arrival execution yields negative IS*, so higher is better. Let p_{ref} denote the reference (arrival) price at the start of the episode, and let $\bar{p}_t(\tilde{a}_t)$ be the volume-weighted execution price obtained by executing the filled quantity $\tilde{a}_t$ (resulting from the submitted action a_t) as a market order by book-walking at time t. Define a side multiplier $m = -1$ for BUY and $m = +1$ for SELL. We use the incremental reward

$$r(s_t, a_t) \ = \ m\, \frac{\tilde{a}_t \big(\bar{p}_t(\tilde{a}_t) - p_{\text{ref}}\big)}{V\, p_{\text{ref}}}, \tag{3}$$

so that with terminal completion (the environment forces $q_{T+1} = 0$ via a final market order [1]), we have $\sum_{t=1}^{T} \tilde{a}_t = V$ and therefore $\sum_{t=1}^{T} r(s_t, a_t)$ equals the episode IS exactly. We report IS in bps; the conventional positive shortfall cost is simply its negative [5].

Almgren-Chriss (AC) framework The AC framework [6] produces a deterministic reference schedule $(AC_1, \ldots, AC_T)$ for executing a parent order of size V over T time steps (so that $\sum_{t=1}^{T} AC_t = V$). Under the standard discrete-time linear-impact specification, AC chooses a schedule by minimising a mean-variance objective for implementation shortfall with explicit parameters $(\sigma, \tau, \eta, \rho, \lambda)$:

$$\min_{(a_t)_{t=1}^{T}} \ \frac{\eta}{\tau} \sum_{t=1}^{T} a_t^2 \ + \ \frac{\rho}{2} V^2 \ + \ \lambda\, \sigma^2 \tau \sum_{t=1}^{T} q_t^2 \quad \text{s.t.} \quad q_{t+1} = q_t - a_t,\ q_1 = V,\ q_{T+1} = 0. \tag{4}$$

We emphasise that the concrete interpretation of impact parameters (e.g., η and ρ) depends on the chosen execution and price impact assumptions. We discuss these assumptions later, where we specify the grounding environment used in our experiments.

Bridging AC framework to RL Following [1], we use an AC trajectory $(AC_1, \ldots, AC_T)$ as a reference schedule and let the RL agent adapt it online via a discretised state representation and a discrete multiplicative action.

State discretisation The continuous market and inventory variables are mapped to a finite state space using a tuple

$$z_n = \langle t_n,\ i_n,\ s_n,\ v_n \rangle, \tag{5}$$

where $t_n \in \{1, \ldots, T\}$ is elapsed time, $i_n \in \{1, \ldots, I\}$ is a bucketed remaining-inventory state, and (s_n, v_n) are percentile-bucketed spread and bid/ask volume states computed from L2 order book snapshots.

Action space (AC multiplier) At each time step t, the agent selects a multiplier $\beta_j \in [\beta_{LB}, \beta_{UB}]$ from a discrete grid with increment β_{incr} and submits the child order

$$a_t^{(j)} = \beta_j\, AC_t, \qquad \beta_j = \beta_{j-1} + \beta_{\text{incr}}. \tag{6}$$

This preserves the AC structure while allowing the policy to trade more or less aggressively than the reference schedule. With a participation cap, the submitted child order $a_t^{(j)}$ may be partially filled. We denote the filled quantity by $\tilde{a}_t^{(j)} \leq a_t^{(j)}$ and update inventory as $q_{t+1} = q_t - \tilde{a}_t^{(j)}$, with terminal completion handled by the environment.

Order execution, price impact, and implementation We designate [1] as the grounding work, which means that we will follow its assumptions about the price process and execution mechanics. This is crucial because, unlike portfolio optimisation where experiments are run on longer time horizons, optimal execution is relatively short-lived and heavily influenced by intraday price impact. In [1], the key assumptions are (1) all orders are market orders, (2) execution is simulated by walking the book up to 5 levels of real order book data, (3) the order book is resilient which means no permanent impact, and (4) remaining orders are automatically treated as a market order at the end of the trading horizon.

On the other hand, [2] specifies an explicit price process with a permanent impact term in a purely simulated environment, and models execution impact by a tunable parameter. [3] instead approximates execution using mid-prices on real data, ignoring the ask-bid spread. As a result, it is not entirely clear that these RL papers are solving the same underlying problem, even if they are all framed as learning optimal execution policies under price impact. [4] also assumes order book resilience, applies a quadratic penalty when trading volume exceeds order book depth 5, and enforces liquidation at last time step.

We emphasise that there is no inherently right or wrong choice among these modelling decisions; however, **such differences in price dynamics and execution mechanics make direct comparisons across papers difficult and often not meaningful**, thereby hindering our goal of examining different modes of reinforcement learning.

As for implementing these assumptions, we are aware that there exists an ecosystem of packages that simulate various market conditions such as FinRL [7] and ABIDES [8]. However, we decided to implement a vanilla Python environment rather than rely on external packages. In our experience, the technical burden and integration overhead of these frameworks can shift the focus toward market microstructure and systems details over reinforcement learning, which would defeat the purpose of this coursework. However, we implement deep RL algorithms by wrapping our environment for Stable Baselines3, a library which implements popular deep RL algorithms through PyTorch [9].

2 Methodology

Our objective is to compare (1) tabular methods, (2) deep value-based methods, and (3) actor-critic methods under an *identical* execution simulator and reward definition. We therefore fix the environment, feature set, action constraints, terminal liquidation rule, and train/test splits across algorithms, varying only the learning rule and exploration mechanism as previously discussed. We briefly cover the formulations of the methodologies along with a summary of their specification.

Tabular Q We first consider classical off-policy TD control with a discrete state-action space, maintaining an explicit action-value table $Q(s, a)$ [10, 11]. Given a transition $(s_t, a_t, r_{t+1}, s_{t+1})$, the update targets the greedy bootstrap value,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big),$$

with exploration typically enforced via an ε-greedy behaviour policy. This baseline is simple and sample-efficient in small, discretised MDPs, and serves as a reference for understanding the effect of off-policy bootstrapping and greedy target selection.

SARSA SARSA is an on-policy counterpart that evaluates and improves the same policy used to generate data [12]. Using the next action sampled from the current policy, $a_{t+1} \sim \pi(\cdot \mid s_{t+1})$, SARSA updates

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\Big).$$

Because its bootstrap target follows the behaviour policy, SARSA can yield more conservative control under exploratory action selection, and provides a natural comparison to off-policy Q-learning when exploration materially affects outcomes.

DDQN To scale value-based control beyond tabular settings, we approximate $Q(s, a)$ with a neural network. Standard DQN-style targets can suffer from overestimation due to the max operator. DDQN mitigates this by decoupling action selection and evaluation [13]. With online parameters θ and target parameters θ^-, the TD target becomes

$$y_t = r_{t+1} + \gamma\, Q\Big(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-\Big),$$

and θ is trained to minimise $\big(y_t - Q(s_t, a_t; \theta)\big)^2$, while θ^- is updated periodically. As covered in the lecture, this typically improves stability and reduces optimistic bias relative to single-network targets.

PPO Finally, we consider a policy-gradient method that directly optimises a parameterised stochastic policy $\pi_\theta(a \mid s)$ with improved update stability [14]. PPO maximises a clipped surrogate objective using the probability ratio $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$,

$$\max_\theta \; E\Big[\min\big(r_t(\theta)\hat{A}_t,\; \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big],$$

typically paired with a learned value baseline to reduce variance (the actor-critic). The clipping mechanism constrains the effective policy step, yielding robust performance across a wide range of continuous and high-dimensional control problems.

3 Experiment

Market data and episode construction We cache 5-minute L2 snapshots for AAPL from Feb 11, 2025 to Feb 10, 2026. Our by-hour experiments restrict episode starts to the fixed UTC grid 14:30–20:30. Our raw data for building the snapshots are based on NASDAQ ITCH-TotalView for order book update events provided by DataBento. Our venue and equity coverage differs from that of [1], but the constructions are identical. Each episode corresponds to a parent order of size V executed over T decision times on an equally spaced grid. At each time step $t \in \{1, \ldots, T\}$, the simulator queries the nearest book snapshot to compute state features and to execute the agent's market order via book-walking up to $L = 10$ levels of the current 5-minute snapshot capped by the maximum participation rate, extending the depth from the original $L = 5$ in [1]. At $t = T$ the environment forces liquidation of any remaining inventory with no cap and allows overflow fills at the worst visible price to enforce completion.

Experiment protocol We split the dataset chronologically 50/50 into a training split and a held-out split. For comparability, we evaluate all methods on fixed episode start-index sets drawn from the held-out split and shared across algorithms (with identical RNG seeds across methods): (1) *by-hour*, where episodes start once per day at each specified UTC time $H \in \{14{:}30, 15{:}30, \ldots, 20{:}30\}$, and (2) *no-hour*, where episode starts are sampled uniformly at random from the entire held-out split. For tabular methods, the by-hour regime trains one policy per H, whereas the no-hour regime trains a single pooled policy. For deep agents, we train one policy per regime: by-hour training restricts episode starts to the union of the hour-specific start indices, while no-hour training samples starts uniformly. Note that by-hour vs no-hour therefore changes the support of start times (and, in our cached data, the no-hour regime also includes extended-hours starts), so differences between regimes conflate time-conditioning with a broader start-time distribution. PPO and DDQN are trained through a Gym wrapper that exposes the discretised state (t, i, s, v) as a normalised 4-vector in $[0, 1]^4$ and uses the same 9-action β grid. Deep agents are evaluated every K environment steps on the fixed held-out evaluation set to produce learning curves, and we select the best configuration in each regime by the pooled median IS on that same set. Therefore, reported medians are slightly optimistic relative to a pure test-only evaluation.

Specification The detailed specification and parameters can be referenced through tables in the Python notebook due to lack of space in the main report. The specification includes baseline settings, experiment settings, parameters for the tabular RL algorithm, and network settings for deep RL algorithms. Note that we kept the network design as simple as possible because our goal is not to hack performance.

4 Results and Analysis

In this section, we state our key findings backed by numerical results. We leave the theoretical interpretation of these results based on the mode of RL to the discussion section.

Replicating the key qualitative result of [1]: tabular improvements concentrate at early/late start times. Figure B.1 provides a direct qualitative check that tabular Q-learning improvements are most noticeable at early and late start times in our by-hour grid (14:30 and 20:30 UTC), especially at $T = 4$. At $T = 4$, SARSA improves over AC by +0.75 bps at 14:30 and +0.82 bps at 20:30, while being negative at some mid-session hours. Q-learning shows the same flavor: it is positive at 14:30 and strongly positive at 20:30, but can be negative mid-session. This aligns with the narrative that intraday regimes differ enough that a single pooled policy without an explicit temporal feature is forced to compromise.

Tabular RL: hour-conditioning is the difference between improvement and regression. According to Figure B.2, for by-hour at $T = 8$, tabular methods show positive median uplifts at several start times, but the gains are not uniform across hours. According to Figure B.3, at $T = 12$ the by-hour tabular results are more mixed across hours, less consistent than at $T = 4/T = 8$. One plausible explanation is that, at longer horizons, the discretised microstructure features provide weaker exploitable signal relative to noise in the aggregated state space.

We now turn to training reward convergence behaviour to understand the agent's learning process. Figure B.7a, B.7b, and B.7c show the pooled (no-hour) tabular policies generally underperforming relative to the baseline at longer horizons (with SARSA at $T = 4$ as a small exception). Pooling across intraday regimes without giving the policy an explicit temporal feature, and evaluating on a broader start-time distribution in the no-hour regime, systematically degrades tabular performance in our setting.

Put differently, the claim from [1] that tabular methods work depends on the aggregation choice. As shown in Table A.1, pooled medians can look consistently negative for tabular RL algorithm, while Figures B.1 and B.2 can make them look positive.

Deep RL does *not* learn a rich state-dependent execution rule. It learns a constant schedule multiplier. In the by-hour regime, the deep agents often match the baseline. At $T = 12$, DDQN collapses exactly to AC by selecting $\beta = 1$. According to Figures B.1, B.2 and B.3 PPO exhibits positive performance at some hours, but it does not exploit the intraday patterns we identified in early and late hours, suggesting it is not responding to order-book-related state variables (unlike the tabular algorithm).

In Table A.2, in the no-hour regime, PPO chooses a single multiplier with 100% frequency, e.g. $\beta = 1$ at $T = 4$, $\beta = 1.25$ at $T = 8$ and $T = 12$. DDQN also chooses a single constant β with 100% frequency, e.g. $\beta = 2.0$ at $T = 4$, $\beta = 1.5$ at $T = 8$, and $\beta = 1.75$ at $T = 12$. **This implies that, for these runs, the deep RL agents are not using the order-book-derived features to adapt execution intensity. Instead, they are effectively learning a one-parameter family of schedules scaled on AC!**

To avoid ambiguity, we distinguish between *deterministic collapse* (a policy uses a single β almost everywhere) and *AC collapse* (deterministic collapse at $\beta = 1$, i.e. exact recovery of the AC schedule). Under this terminology, PPO exhibits AC collapse only in the no-hour regime at $T = 4$, while DDQN exhibits AC collapse in the by-hour regime at $T = 12$ (and is nearly AC-collapsed at $T = 8$).

We stress once again that this observation matters for interpretation. If the learned deep policy is nearly a constant multiplier, a plausible explanation is that any improvement

over AC is driven primarily by *aggressiveness tuning* under our simulator's completion mechanics, rather than by extracting predictive signals from (s, v).

Deep RL agents shine at longer horizons via variance and tail-risk control. In general, deep RL-related optimal execution research focuses on exploiting deep order book level features to drive performance [2, 3, 4]. This makes sense because deep learning algorithms excel in extracting features from unstructured data. From this perspective, the state-action space of [1], in theory, should be too simple for deep-learning-based algorithms to offer meaningful improvements due to heavy quantisation. Surprisingly, the deep RL algorithm does show an edge over the tabular algorithm on longer horizons by exploiting other aspects of the problem.

Although deep RL does not appear to learn strongly state-dependent policies, it can still improve the distribution of outcomes. This is clearest at $T = 12$ in the no-hour regime. In Table A.1, relative to AC with median ≈ -4.50 bps, PPO improves the pooled median by about $+0.42$ bps and DDQN improves it by about $+0.44$ bps. According to Figure B.7c, PPO's median improves rapidly from a poor untrained policy at step 0 to its plateau by around 100k environment steps, after which changes are marginal. DDQN's curve is more variable across checkpoints, but the selected policy still ends up as a simple constant-β schedule.

In Table A.3, comparing no-hour quantiles at $T = 12$, AC has a 10th percentile around -21.2 bps and a 90th percentile around $+14.7$ bps. PPO shifts these to about -18.4 bps (10th) and $+13.0$ bps (90th), and DDQN shifts them further to about -16.4 bps (10th) and $+9.7$ bps (90th). This is a narrowing of the distribution: the policies give up some upside but materially improve the downside tail. At $T = 8$, the same pattern appears in quantiles and standard deviations but does not translate into a median uplift. At $T = 4$, DDQN improves the median by $+0.27$ bps while PPO collapses to the AC schedule exactly.

Why aggregation choices change the story. Taken together, we state our thesis: ***RL algorithm comparisons in optimal execution are highly sensitive to seemingly minor protocol choices.*** If we summarise by per-hour medians after training per-hour tabular policies, tabular Q-learning looks consistently better than AC. If we instead pool across hours and train a single policy without an explicit hour-of-day feature, tabular methods can look materially worse than AC at longer horizons. Deep RL flips the emphasis again: its best pooled results at $T = 12$ are consistent with learning a near-deterministic schedule multiplier. We hypothesise that this reduces tail risk induced by the terminal completion rule.

5 Discussion

In this section, we interpret the results of the previous section based on our theoretical understanding of RL algorithms.

On-policy vs off-policy: target comparison. Q-learning and SARSA differ only by their bootstrap target.

$$\text{Q-learning: } r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') \quad \text{vs} \quad \text{SARSA: } r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}),$$

with a_{t+1} sampled from the current ε-greedy behaviour policy. We can interpret the max operator as the culprit of larger variance compared to baselines in $T = 8, 12$ because it forces tabular Q-learning to chase short-term greedy actions. This is consistent with the idea mentioned during the lecture that off-policy algorithms tend to exhibit larger variance. SARSA's on-policy target can be understood as relatively conservative under exploration because it evaluates the value of actual behaviour policy. This idea is observed in the pooled summaries at $T = 4$ where SARSA has lower variance compared to Q-learning.

However, similar patterns cannot be identified between PPO (on-policy) and DDQN (off-policy) because both policies show mode collapse.

Tabular vs Deep RL: aliasing vs generalisation. The advantage of tabular RL is that it could exactly memorise events with discrete bins. This means that no function approximation is required if a specific state corresponds to a unique rewarding action. However, this can backfire if different regimes get mapped into the same bin (aliasing effect). This is clearly shown by our by-hour vs no-hour regime split. We clearly observed that training separate policies over hours looks stronger than all hours pooled together. Mathematically speaking, this means that $Q(z, a)$ is sampling from a mixture distribution for the no-hour regime, which distorts Bellman targets between mixtures.

On the other hand, the deep RL algorithm uses function approximation to generalise across states over smooth noisy bins. This is usually an advantage, but acts as a disadvantage for this specific formulation of optimal execution because the microstructural signal in (t, i, s, v) can be weak compared to structural risk such as terminal completion risk. Since our action space is extremely coarse with AC acting as a strong baseline, the easiest way for a deep RL agent to avoid this risk is to "cheat" by discovering a single action almost everywhere. ***So the seemingly bizarre and lazy behaviour of deep RL agents could be understood as exploiting the execution pattern of our simulator, by identifying a simple and stable solution***.

Actor-critic vs value-based: Why PPO and DDQN collapse to deterministic constant-β policies. PPO is an on-policy actor-critic method optimising a clipped surrogate objective. Its policy gradient term is

$$\nabla_\theta J(\theta) = E_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\Big].$$

meaning that PPO pushes probability mass towards actions with positive advantage $\hat{A}_t$. In practice, $\hat{A}_t$ measures how much better the realised return is relative to the value baseline $V_\phi(s_t)$ (e.g. $\hat{A}_t \approx G_t - V_\phi(s_t)$, or via GAE). So our observed behaviour can be understood as this mechanism concentrating probability mass onto a single constant multiplier β that tends to achieve higher returns under our simulator, and eventually stabilising due to the clipping mechanism and low entropy regularisation.

DDQN collapses as well, but it does so by learning action-value and selecting greedily:

$$y_t = r_{t+1} + \gamma\, Q_{\text{target}}\left(s_{t+1}, \arg\max_{a'} Q_{\text{online}}(s_{t+1}, a')\right), \qquad \pi(s) = \arg\max_a Q_{\text{online}}(s, a).$$

So DDQN collapses not due to a probability-mass push like PPO, but when the network learns that a particular constant β has the highest $Q(s, a)$ for most states. In our environment, it can converge to $\beta > 1$, which is consistent with selecting a more aggressive

constant multiplier that maximises expected return under forced terminal completion, thereby reducing downside-tail outcomes. This algorithmic difference helps explain why PPO and DDQN can collapse to different constant multipliers under the same simulator.

Theoretical coherence of deep RL agents. Once we frame IS uncertainty as inventory risk, the variance reduction becomes almost inevitable. In AC-style models, the risk term is proportional to $\sum_t q_t^2$, so any policy that front-loads execution reduces inventory exposure and outcome variance [6]. Delaying execution means larger variance due to volatility risk. In our environment, the forced terminal liquidation amplifies this: leaving inventory to the final step creates large left-tail events when depth/spread are unfavorable. Deep agents that converge to a consistently more aggressive $\beta > 1$ effectively implement a crude but risk-reducing rule: "liquidate earlier, accept more impact now, avoid terminal disaster later." That mechanism can improve the median even if mean improvements are small or mixed, which matches the narrative we see at longer horizons in the pooled regime.

Coherence with other work. [1] reports that tabular Q-learning yields the largest gains at the start and end of the trading day, and that these gains weaken as the execution horizon lengthens. We observe the same pattern in our experiments. Subsequent deep-RL optimal execution literature tends to focus on shorter time horizons and higher-frequency market data [2, 3, 4]. These papers do not explore tabular RL, so we used it as an opportunity to provide a more holistic coverage of RL algorithms under the optimal execution setting. [4] implements [3], but their focus is on driving performance by leveraging neural network architectures such as RNNs to exploit temporal features. Our setting could be understood as being too simple for a neural network to identify meaningful intraday microstructural patterns, given the heavily binned and discretised order book features. This led us to discover that deep RL agents may rely less on state-contingent microstructure and more on exploitable simulator regularities. We find the results in [2] similar in the sense that DDQN is validated by testing whether it can discover a hidden but known optimal strategy in the simulator. More broadly, our results highlight that reported deep-RL improvements in execution are sensitive to simulator assumptions and to the richness of the state–action design, which makes controlled, like-for-like comparisons across algorithm families informative.

6 Conclusion

We re-implemented the Hendricks–Wilcox optimal execution environment on AAPL L2 data and compared tabular Q-learning, SARSA, DDQN, and PPO under a fixed simulator and reward. Tabular methods can outperform tuned AC in the by-hour regime, but pooled no-hour training degrades tabular performance at longer horizons, consistent with state aliasing across intraday regimes (and a broader start-time distribution). In contrast, PPO and DDQN frequently converge to near-deterministic constant-β schedules, sometimes exactly reproducing AC ($\beta = 1$), so their gains are most plausibly explained by aggressiveness tuning and tail-risk control under the terminal completion rule. Overall, the apparent ranking of RL algorithms in optimal execution is highly sensitive to protocol choices such as time conditioning, pooling, and completion mechanics.

A Tables

T	Method	No-hour	
		Median (bps)	Var (bps^2)
$T = 4$			
4	AC_TUNED	-5.331	117.3
4	TWAP	-5.362	129.6
4	Q-learning	-5.450	142.5
4	SARSA	-5.271	120.0
4	PPO	-5.331	117.3
4	DDQN	-5.066	96.6
$T = 8$			
8	AC_TUNED	-4.793	334.9
8	TWAP	-4.778	365.9
8	Q-learning	-5.947	598.5
8	SARSA	-5.797	463.9
8	PPO	-4.806	257.6
8	DDQN	-4.830	201.6
$T = 12$			
12	AC_TUNED	-4.501	3729.1
12	TWAP	-4.479	3737.8
12	Q-learning	-5.238	4058.3
12	SARSA	-5.055	3935.2
12	PPO	-4.079	3642.3
12	DDQN	-4.060	3523.9

Table A.1: No-hour test-set median and variance of (signed) implementation shortfall (IS, bps) for $V = 3000$ (higher is better; negative means worse-than-arrival execution). Variance is computed as sample variance of per-episode IS (bps). Run: `full_20260227_232247`.

T	Regime	Method	β_{mode}	Mode (%)	# β used
$T = 4$					
4	By-hour	PPO	1.5	53.4	2
4	By-hour	DDQN	0.75	100.0	1
4	No-hour	PPO	1	100.0	1
4	No-hour	DDQN	2	100.0	1
$T = 8$					
8	By-hour	PPO	1.5	57.7	3
8	By-hour	DDQN	1	98.5	3
8	No-hour	PPO	1.25	100.0	1
8	No-hour	DDQN	1.5	100.0	1
$T = 12$					
12	By-hour	PPO	1.5	100.0	1
12	By-hour	DDQN	1	100.0	1
12	No-hour	PPO	1.25	100.0	1
12	No-hour	DDQN	1.75	100.0	1

Table A.2: Deep RL action concentration for $V = 3000$, computed from the best selected PPO/DDQN policies in run `full_20260227_232247`. For each $(T, \text{regime}, \text{method})$, we report the most frequent action multiplier β_{mode}, its usage frequency (Mode %), and the number of distinct β values used with nonzero count. By-hour pools action counts across all by-hour evaluation episodes (all start times H); no-hour uses the fixed no-hour evaluation set.

T	Regime	Method	q_{10}	Median	q_{90}	Std
$T = 4$						
4	By-hour	AC_TUNED	-11.7	-1.2	9.4	13.8
4	By-hour	PPO	-10.2	-1.2	7.9	13.2
4	By-hour	DDQN	-13.2	-1.3	11.6	17.2
4	No-hour	AC_TUNED	-17.9	-5.3	5.7	10.8
4	No-hour	PPO	-17.9	-5.3	5.7	10.8
4	No-hour	DDQN	-17.0	-5.1	2.8	9.8
$T = 8$						
8	By-hour	AC_TUNED	-18.3	-0.7	17.3	23.1
8	By-hour	PPO	-13.9	-0.7	13.1	17.9
8	By-hour	DDQN	-18.3	-0.8	17.3	23.0
8	No-hour	AC_TUNED	-20.6	-4.8	10.1	18.3
8	No-hour	PPO	-19.7	-4.8	8.7	16.0
8	No-hour	DDQN	-18.9	-4.8	7.2	14.2
$T = 12$						
12	By-hour	AC_TUNED	-25.0	-0.7	21.4	28.2
12	By-hour	PPO	-18.8	-0.6	17.9	23.8
12	By-hour	DDQN	-25.0	-0.7	21.4	28.2
12	No-hour	AC_TUNED	-21.2	-4.5	14.7	61.1
12	No-hour	PPO	-18.4	-4.1	13.0	60.4
12	No-hour	DDQN	-16.4	-4.1	9.7	59.4

Table A.3: Distribution summary of per-episode implementation shortfall (IS, bps) for $V = 3000$ in run `full_20260227_232247`: 10th percentile (q_{10}), median, 90th percentile (q_{90}), and sample standard deviation (Std). By-hour pools all by-hour evaluation episodes across start times; no-hour uses the fixed no-hour evaluation set.

B Figures

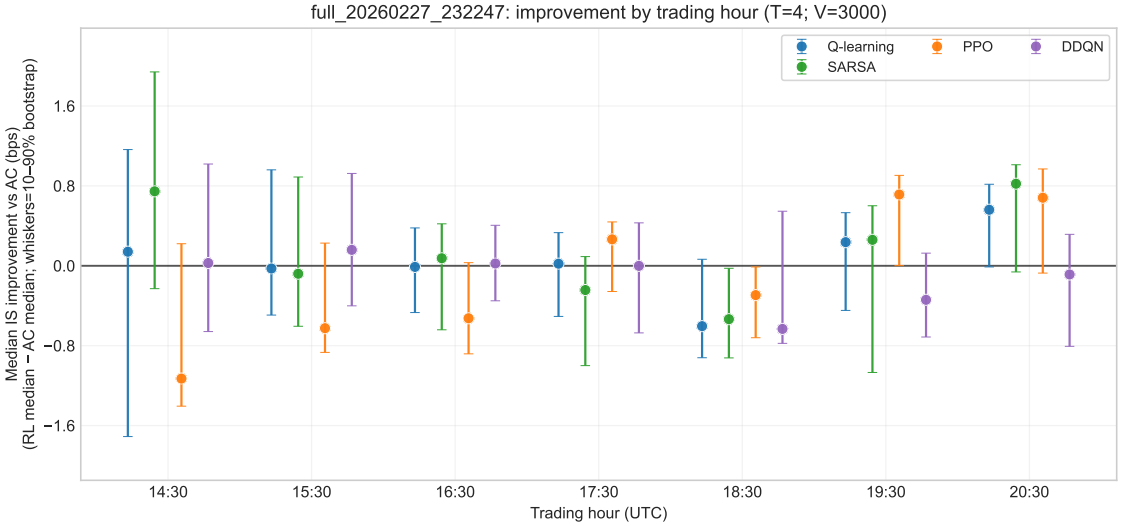


Figure B.1: Median IS improvement relative to AC_TUNED by start hour (by-hour evaluation), $T = 4$

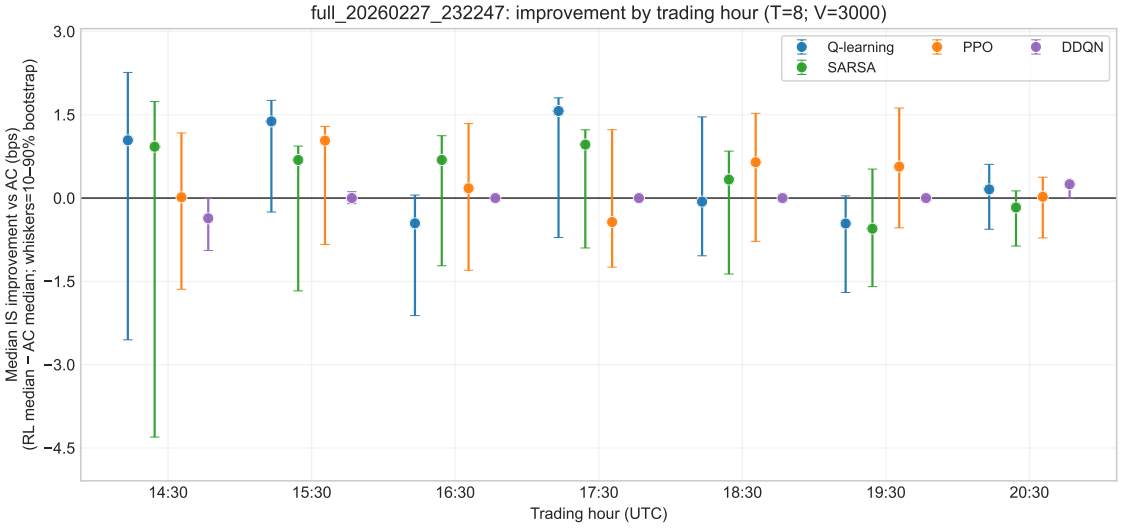


Figure B.2: Median IS improvement relative to AC_TUNED by start hour (by-hour evaluation), $T = 8$

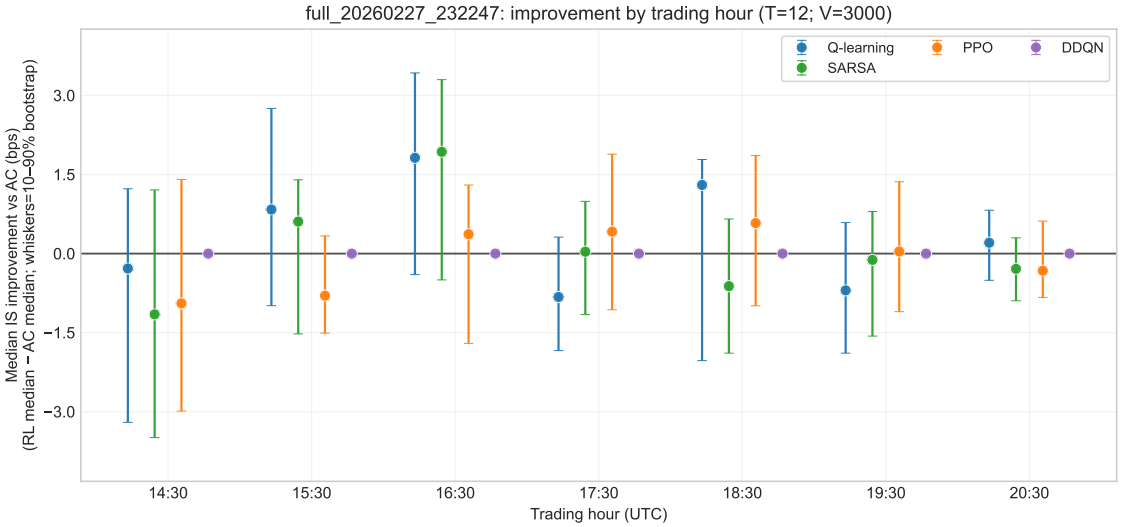


Figure B.3: Median IS improvement relative to AC_TUNED by start hour (by-hour evaluation), $T = 12$

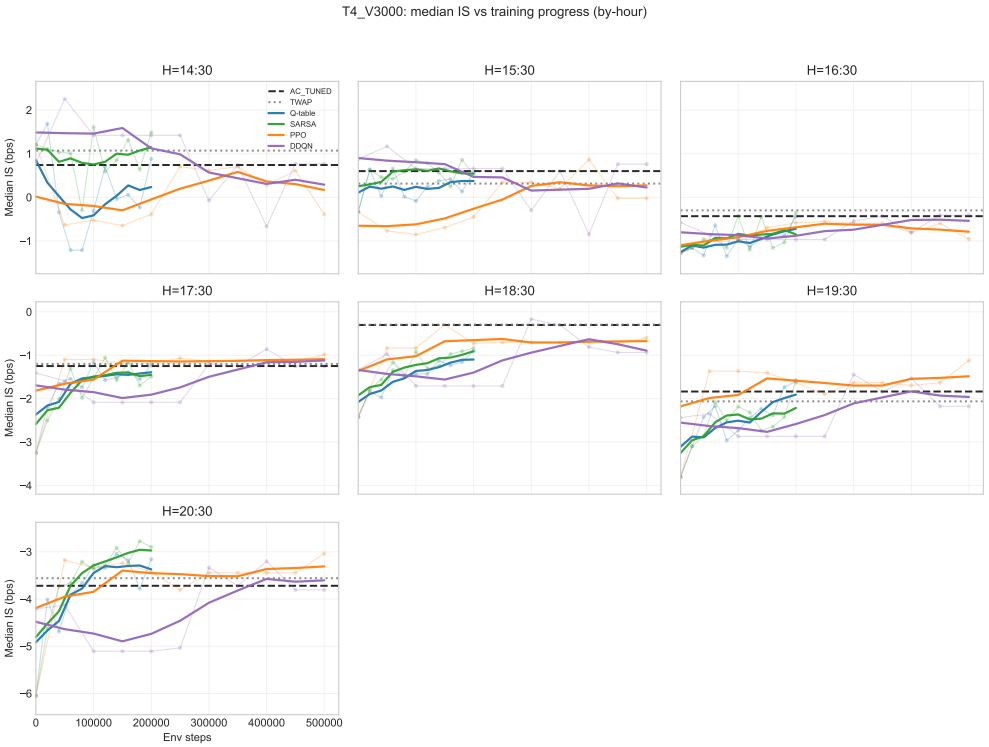


Figure B.4: Median IS vs training progress (by hour), $T = 4$

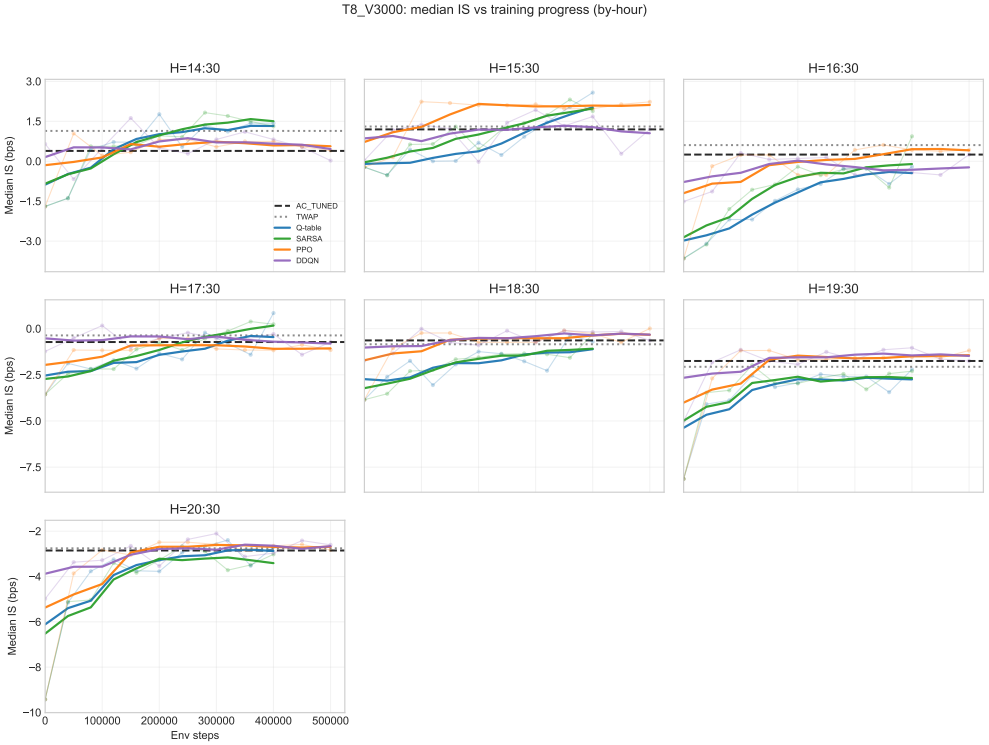


Figure B.5: Median IS vs training progress (by hour), $T = 8$

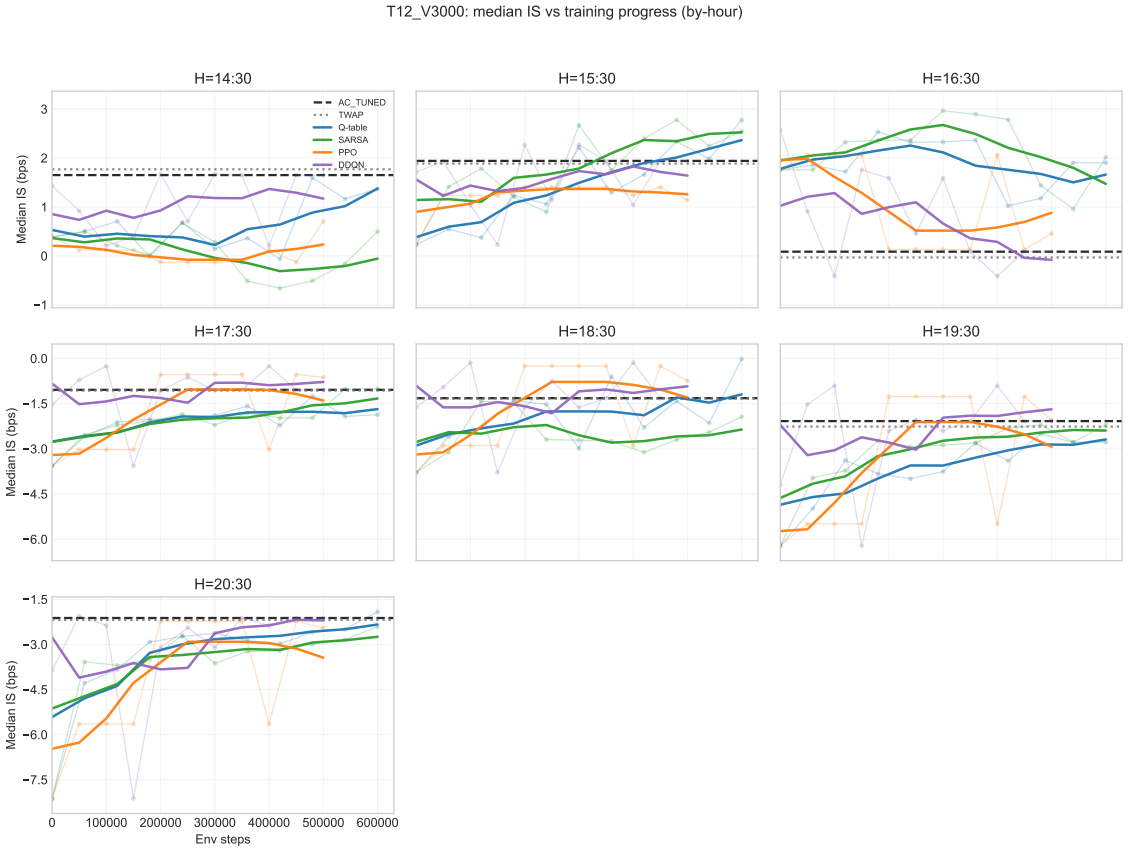
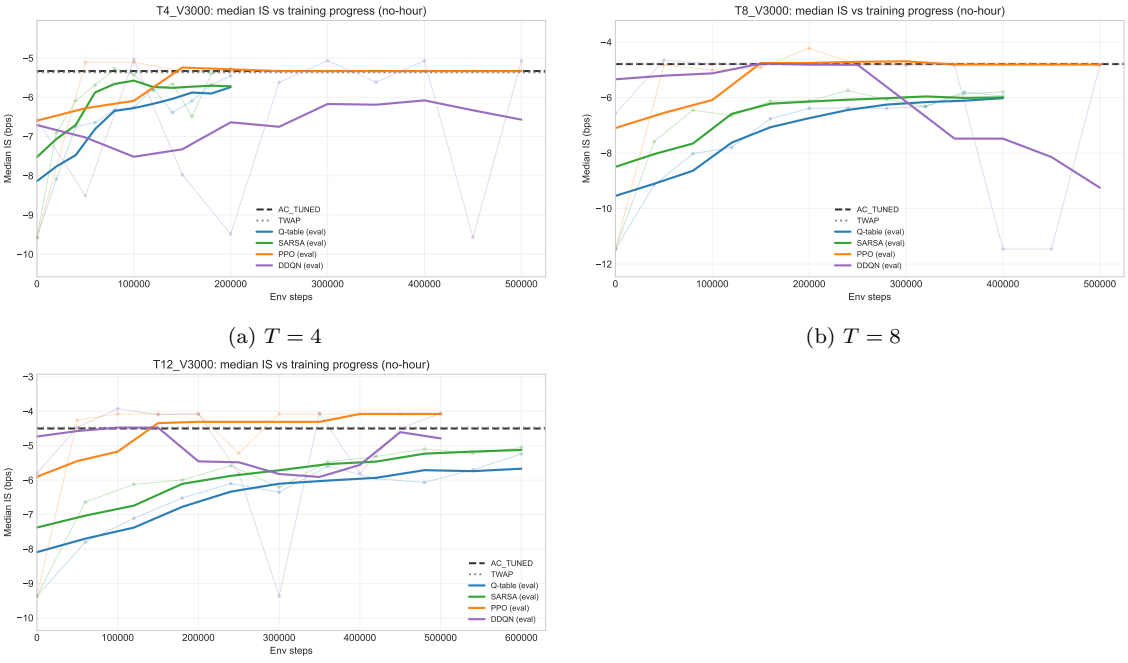


Figure B.6: Median IS vs training progress (by hour), $T = 12$



(a) $T = 4$

(b) $T = 8$

(c) $T = 12$

Figure B.7: Median IS vs training progress (no-hour training regime) for $T \in \{4, 8, 12\}$.

References

[1] Dieter Hendricks and Diane Wilcox. "A Reinforcement Learning Extension to the Almgren-Chriss Model for Optimal Trade Execution". In: *2014 IEEE Conference on Computational Intelligence for Financial Engineering & Economics (CIFEr)*. Also available as arXiv:1403.2229. IEEE, Mar. 2014, pp. 457–464.

[2] Kevin Dabérius, Elvin Granat, and Patrik Karlsson. "Deep Execution - Value and Policy Based Reinforcement Learning for Trading and Beating Market Benchmarks". In: *SSRN Electronic Journal* (Apr. 2019).

[3] Brian Ning, Franco Ho Ting Lin, and Sebastian Jaimungal. "Double Deep Q-Learning for Optimal Execution". In: *Applied Mathematical Finance* 28.4 (July 2021), pp. 361–380.

[4] Siyu Lin and Peter A. Beling. "An End-to-End Optimal Trade Execution Framework based on Proximal Policy Optimization". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*. International Joint Conferences on Artificial Intelligence Organization, July 2020, pp. 4548–4554.

[5] Ben Hambly, Renyuan Xu, and Huining Yang. "Recent Advances in Reinforcement Learning in Finance". In: *arXiv preprint* arXiv:2112.04553 (Mar. 2023).

[6] Robert Almgren and Neil Chriss. "Optimal Execution of Portfolio Transactions". In: *Journal of Risk* 3.2 (Jan. 2001), pp. 5–39.

[7] Xiao-Yang Liu et al. "FinRL: Deep Reinforcement Learning Framework to Automate Trading in Quantitative Finance". In: *Proceedings of the 2nd ACM International Conference on AI in Finance (ICAIF '21)*. Association for Computing Machinery, Nov. 2021.

[8] David Byrd, Maria Hybinette, and Tucker Balch. "ABIDES: Towards High-Fidelity Multi-Agent Market Simulation". In: *Proceedings of the SIGSIM Principles of Advanced Discrete Simulation (SIGSIM-PADS '20)*. Association for Computing Machinery, June 2020, pp. 11–22.

[9] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[10] Christopher J. C. H. Watkins and Peter Dayan. "Q-learning". In: *Machine Learning* 8.3 (May 1992), pp. 279–292.

[11] Christopher John Cornish Hellaby Watkins. "Learning from Delayed Rewards". PhD thesis. Cambridge, UK: King's College, University of Cambridge, May 1989.

[12] G. A. Rummery and M. Niranjan. *On-Line Q-Learning Using Connectionist Systems*. Tech. rep. CUED/F-INFENG/TR 166. Cambridge, England: Cambridge University Engineering Department, Sept. 1994.

[13] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-Learning". In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI-16)*. AAAI Press, Feb. 2016, pp. 2094–2100.

[14] John Schulman et al. *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347. July 2017.

Acknowledgments

We acknowledge the use of **GPT-5.2** for assistance in generating code for figures and plots, as well as for LaTeX formatting of tables and results.